

MAGIC: Detecting APTs

SUBMITTED BY:

PADMESH SHUKLA 2023UCP1842

ASIT YADAV 2023UCP1588



Overview

Understanding APTs

APTs are sophisticated cyber-attacks that are intentionally conducted by highly skilled attackers.

They utilize stealthy behaviors that change over time, making them exceptionally difficult to detect with traditional security rules.

They frequently exploit zero-day vulnerabilities that existing defense systems are often not yet trained to handle or recognize.

Impact of APTs

Accurately recovering an “attack story” and identifying key malicious system entities is vital but difficult, often requiring redundant manual efforts and high-level domain expertise. Frequent false alarms caused by “concept drift” (natural changes in system behavior over time) often tire and confuse security analysts, potentially leading to missed detections. Enterprises generate hundreds of GB of audit logs daily, creating massive computation and memory overhead.

PROBLEM STATEMENT AND CURRENT RISKS

- The increasing sophistication and adaptability of APT campaigns has rendered traditional security mechanisms largely ineffective. Modern attack toolkits employ anti-forensics, living-off-the-land techniques, and encrypted command-and-control channels to blend seamlessly into legitimate enterprise traffic.
- Current Intrusion Detection Systems are constrained to catalogued threats; zero-day exploits are entirely invisible. Enterprise activity volumes cause alert fatigue, masking genuine threats among noise.
- Sequential log parsing discards structural and temporal relationships between entities. Real-time analysis of millions of daily audit events exceeds deployed system capacity.

Provenance Graphs in Cybersecurity Monitoring

1. **Audit-to-Graph Transformation:** Converts massive, unstructured system audit logs into a directed cyclic graph that maps every system event.
2. **System Entities (Nodes):** Represents the "actors" in a system, including running Processes, FileObjects, and NetFlowObjects (network sockets).
3. **Interactions (Edges):** Models the actions taken between entities, such as a process executing a file, reading data, or connecting to an IP address.
4. **Fine-Grained Causality Analysis:** Provides the "breadcrumb trail" needed to trace an attack back to its entry point, such as a phishing email or a zero-day exploit.
5. **Contextual Semantics:** Captures deep correlations between benign activities that traditional, rule-based systems often overlook.



MAGIC Overview and Architecture



The architecture consists of 4 parts:

1. Provenance Graph Construction (pre processor)

2. Graph Representation Module

- Feature Masking
- Graph Encoder
- Graph Decoder

3. Detection Module



Provenance Graph Construction (pre processor)

1. Log Parsing: Turning Text into a Map

The first goal is to extract the basic "who" and "what" from the system logs.

- Extraction: Each log entry is parsed to identify system entities (which become nodes) and interactions (which become edges).
- Categorical Labeling: MAGIC extracts labels for these entities and interactions. If the log format is simple, it uses existing labels; if it is complex, it uses multi-label hashing (like xxhash) to transform detailed attributes into consistent labels.
- Multi-Graph State: At this point, the result is a directed multi-graph, meaning there can be many different arrows (edges) connecting the same two points (nodes).

2. Initial Embedding: Translating Labels to Numbers

Since the AI cannot "read" text labels like "read" or "process," MAGIC must convert them into a mathematical language.

- **Fixed-Size Vectors:** Every unique label is mapped to a feature vector of dimension d .
- **Lookup Mapping:** MAGIC uses a lookup embedding to ensure that identical labels (e.g., two different "read" actions) are assigned the exact same numeric vector, while different labels (e.g., "read" vs. "write") get distinct vectors.
- **Transductive Setting:** Because the possible labels are determined by the audit log format, the model does not need to "invent" embeddings for unseen labels; it simply uses the map it created during setup.

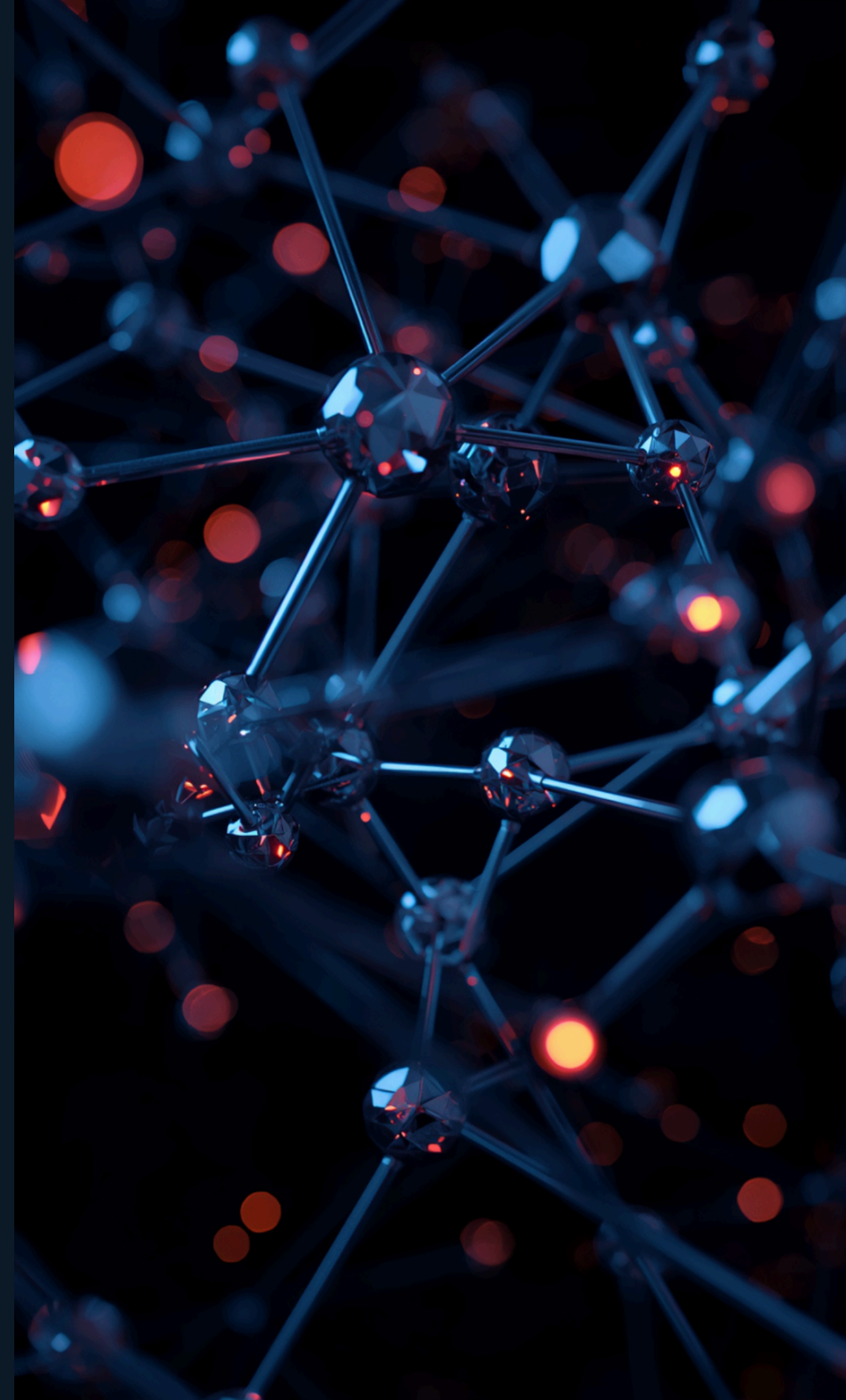
Noise Reduction: The "Mild" Optimization

- **Combining Edges:** The multi-graph is converted into a simple-graph by merging multiple interactions between the same two nodes into one unique edge.
- **Averaging Semantics:** If a pair of nodes has multiple types of interactions (like "read" and "write"), the final single edge is assigned an embedding that is the average of all the combined initial embeddings.
- **Preserving Context:** Unlike other methods that might delete "irrelevant" nodes, MAGIC uses a "mild" approach. It keeps all nodes to ensure the AI has the full context of benign behavior, but it collapses the repetitive arrows to save memory.

Feature Masking in GAT-based Encoder

This masking doesn't happen just once; it is a dynamic part of the self-supervised learning strategy.

- **Random Selection:** During every training round, different nodes are masked to ensure the model practices on every possible part of the graph.
- **Edges are Never Masked:** MAGIC intentionally keeps all the "arrows" (edges) visible. This is because the relationships—who is talking to whom—are the vital clues the model needs to solve the "blank".
- **To correctly reconstruct (guess) a masked node,** the encoder must learn the "unwritten rules" of the system—like the fact that a process talking to a specific system library and a public IP is likely a network tool.



Graph Encoder

The Graph Encoder is the part of MAGIC's "Brain" that processes the cleaned map to understand the deep relationships between system activities.

- **Neighborhood Aggregation:** The encoder doesn't just look at a node in isolation; it looks at its immediate neighbors (one-hop neighborhood).
- **Message Passing:** Information about a "source" node and the "interaction" (the arrow) is packaged into a message and sent to the "destination" node.
- **The Attention Mechanism:** This is the "smart" part. Instead of treating all neighbors equally, GAT uses a mathematical attention coefficient to measure how important each neighbor is.

To catch sophisticated attackers who hide their tracks through multiple steps, MAGIC doesn't just look at the next-door neighbor.

- **Stacking Layers:** MAGIC typically uses 3 layers of GAT.
- **Wider Vision:** Each additional layer allows the encoder to see one step further.
 1. Layer 1 sees immediate neighbors.
 2. Layer 2 sees "friends of friends."
 3. Layer 3 understands the behavior of an entity within its 3-hop neighborhood.
- **Final Node Embedding:** The final "personality" of a node is a combination of its original features and everything it learned from all three GAT layers.
- **Output:** It produces two types of "Numeric Personalities" (embeddings): i) Node Embeddings: For specific entities like a process or file. , ii) System State Embedding: A summary of the entire system's health at that moment.

Graph Decoder

The decoder tries to "guess" the hidden information of the masked nodes.

It uses Masked Feature Reconstruction (guessing the node's properties) and Sample-based Structure Reconstruction (guessing if two nodes should be connected) to calculate a "Loss" score.

This feedback helps the Brain improve its understanding of "normal" behavior.



Detection Module

Once the "Brain" is trained on normal behavior, it can spot attackers in an unsupervised way (meaning it doesn't need to see an attack first to recognize it).

- **Memory Bank (K-D Tree):** MAGIC stores all "normal" personalities it learned during training in a highly organized data structure called a K-D Tree. This allows for lightning-fast searching.
- **The "Weirdness" Test (KNN):** When a new log arrives, MAGIC finds its K-Nearest Neighbors in the memory bank.
- **Anomaly Scoring:** It calculates an anomaly score based on the distance to these neighbors.

Benign: High similarity to known safe behaviors (Low score).

Malicious: Far away from anything seen before (High score).

RESULTS

Granularity	Dataset	Recall	False Positive Rate	Precision	F1-Score	AUC
Batch	Streamspot	100.00%	0.59%	99.41%	99.71%	99.95%
	Unicorn Wgot	96.00%	2.00%	98.02%	96.98%	96.32%
Entity	E3-Trace	99.98%	0.09%	99.17%	99.57%	99.99%
	E3-THEIA	99.99%	0.14%	98.23%	99.11%	99.87%
	E3-CADETS	99.77%	0.22%	94.40%	97.01%	99.77%

Figure 8.1. Official Author Benchmark Results across All Datasets and Granularities

RESULTS

Metric	Value
AUC	0.9998
F1 Score	0.9957
Precision	0.9917
Recall	0.9998
True Positives (TP)	68,072
True Negatives (TN)	615,452
False Positives (FP)	569
False Negatives (FN)	14

Table 8.2. Detailed Performance Metrics — TRACE Dataset

Metric	Value
AUC	0.9987
F1 Score	0.9911
Precision	0.9823
Recall	0.9999
True Positives (TP)	25,318
True Negatives (TN)	318,992
False Positives (FP)	456
False Negatives (FN)	1

Table 8.3. Detailed Performance Metrics — THEIA Dataset

RESULTS

Metric	Value (Mean $\pm$ Std)
AUC	0.9995 $\pm$ 0.0007
F1 Score	0.9954 $\pm$ 0.0065
Precision	0.9920 $\pm$ 0.0113
Recall	0.9990 $\pm$ 0.0070
True Positives (%)	99.90 $\pm$ 0.70
True Negatives (%)	99.18 $\pm$ 1.17
False Positives (%)	0.82 $\pm$ 1.17
False Negatives (%)	0.10 $\pm$ 0.70

Table 8.4. Detailed Performance Metrics — StreamSpot Dataset

Metric	Value (Mean $\pm$ Std)
AUC	0.9739 $\pm$ 0.0190
F1 Score	0.9436 $\pm$ 0.0224
Precision	0.9139 $\pm$ 0.0434
Recall	0.9776 $\pm$ 0.0317
True Positives	24.44 $\pm$ 0.79
True Negatives	22.62 $\pm$ 1.30
False Positives	2.38 $\pm$ 1.30
False Negatives	0.56 $\pm$ 0.79

Table 8.5. Detailed Performance Metrics — WGET Dataset

NOVELTY WORK

- Contribution 1: Degree-Weighted Masking

Our Approach: Degree-Weighted Masking : We replace the uniform random 50% masking proposed in the original paper with a Degree-Weighted Masking scheme. Nodes are sampled for masking with probability proportional to their degree

- Contribution 2: Forensic Decomposition Layer

Our Forensic Decomposition Layer mathematically deconstructs the total reconstruction error into two semantically distinct, independently interpretable components : i) Behavioral : Deviation in node attribute space , ii) Structural : Deviation in graph topology

NOVELTY WORK SAMPLE RESULT

🔥 ANOMALOUS KILL CHAIN RECONSTRUCTION (XAI-OPTIMIZED) 🔥

=====

🚨 PATIENT ZERO IDENTIFIED: /home/admin/pine/pine4.64/pine

[Anomaly Score: 7.22]

↳ 🔵 Forensic Breakdown: 3.9% Behavioral | 🌐 96.1% Structural

↓ Lateral movement to

🦟 Unknown System Entity (Node 183)

[Anomaly Score: 0.00]

↳ 🔵 Forensic Breakdown: 100.0% Behavioral | 🌐 0.0% Structural

THANK YOU

THANK YOU